SYSTEM ANALYSIS DOCUMENTATION (SAD)

FranchiseIQ

AI Decision Intelligence Dashboard for Retail Franchise Management

UMHackathon 2026
umhackathon@um.edu.my

© UMHackathon 2026

Table of Contents

1. Introduction 3
1.1. Purpose 3
1.2. Background 3
1.3. Target Stakeholders 4
2. System Architecture & Design 5
2.1. High-Level Architecture 5
2.2. LLM as Service Layer 5
2.3. Technological Stack 7
2.4. Key Data Flows 7
3. Functional Requirements & Scope 10
3.1. Minimum Viable Product (MVP) 10
3.2. Non-Functional Requirements (NFRs) 10
3.3. Out of Scope / Future Enhancements 11
4. Monitor, Evaluation, Assumptions & Dependencies 12
4.1. Technical Evaluation 12
4.2. Monitoring 13
4.3. Assumptions 13
4.4. External Dependencies 13
5. Project Management & Team Contributions 15
5.1. Project Timeline 15
5.2. Team Members and Roles 15
5.3. Recommendations for Future Development 16
6. Quality Assurance Testing Documentation (QATD) 18
6.1. Objective 18
6.2. Scope and Requirements Traceability 18
6.3. Risk Assessment and Mitigation Strategy 18
6.4. Test Environment and Execution Strategy 18
6.5. Test Case Specifications 19
6.6. AI Output and Boundary Testing 19

1. Introduction

This section introduces the strategic solution for the problem statement identified and framed in the Product Review Documentation. FranchiseIQ is designed as a modular, AI-native platform that bridges the gap between high-volume retail transaction data and executive decision-making at the store level. Rather than positioning artificial intelligence as a decorative feature, the system treats the GLM reasoning layer as a first-class architectural component that can be independently evaluated, upgraded, or removed without breaking the underlying CRUD operations of the platform.

1.1. Purpose

This System Analysis Document (SAD) highlights the technical scope and the key design decisions made throughout the development of FranchiseIQ. The core elements covered in this document are as follows.

The Architecture section describes how FranchiseIQ is structured as a web-based client-server system using a Hexagonal (Ports and Adapters) architecture on the backend, paired with an event-driven data ingestion layer powered by Apache Kafka. The system is designed to ensure that domain logic remains isolated from external frameworks, making it easier to swap out AI providers or messaging infrastructure without restructuring the core application.

The Data Flows section explains how raw Point-of-Sale (POS) transaction data moves from physical store terminals, through the Kafka pipeline, into the Spring Boot application layer, gets processed and stored in PostgreSQL, and finally surfaces as enriched insights on the React dashboard. The AI inference layer intercepts specific requests along this flow and enriches them with structured recommendations before passing results back to the frontend.

The Model Process section walks through the end-to-end workflow of the four core features, namely Dynamic Pricing, Sentiment Analysis, Inventory Forecasting, and Actionable Insights. Each feature follows the same agentic prompting pattern: store context is assembled, injected into a structured prompt template via LangChain4j, sent to the Z AI GLM API, and the raw JSON response is validated and parsed into a flattened DTO before reaching the React component.

The Role of Reference section establishes that this document serves as the primary technical reference for all developers, testers, and evaluators involved in reviewing FranchiseIQ during UMHackathon 2026. It defines the system boundaries, documents the AI integration clearly, and provides traceability between the functional requirements and their implementations.

1.2. Background

For years, franchise management in Malaysia has followed a slow, linear process. Store POS data would be uploaded at the end of each business day, reviewed by a regional analyst the following week, and pricing or inventory decisions would then be distributed to store managers via PDF or email. This cycle typically spans five to ten business days, during which the store has no mechanism to respond to hyper-local events such as a national holiday, a heatwave, a viral product trend, or a competitor price drop.

FranchiseIQ was conceived specifically to address this cycle. The team identified MiXue outlets in Malaysia as a representative target, as they operate in what the team refers to as "sinking markets," which are Tier 2 and Tier 3 cities where supply chain fragility, fragmented data silos, and non-technical store managers make manual decision-making particularly costly. According to the team's initial scoping, product waste runs at approximately 23% due to inaccurate forecasting, and reactive store management rather than proactive management is the norm.

The previous state of the system had no centralised intelligence layer. POS data, customer reviews, weather signals, and audit logs existed in separate, disconnected silos with no automated mechanism to correlate them. The new capabilities introduced in FranchiseIQ include real-time Kafka-based data ingestion, a Z AI GLM reasoning engine integrated through LangChain4j, a human-in-the-loop pricing adjustment interface, and an anti-hallucination framework that validates every AI output against a strict schema before it is displayed to the manager.

1.3. Target Stakeholders

The following table outlines the primary stakeholders of FranchiseIQ, their respective roles within the system, and their key expectations from the platform.

Stakeholder	Roles	Expectations
Store Manager	Adjusts trade-off weights for pricing, reviews AI-generated insights, and monitors daily KPIs via the dashboard.	Real-time feedback on pricing changes, a simple and explainable UI, and AI decisions they can override or understand.
Regional Supervisor	Monitors store health across multiple outlets, reviews sentiment trends, and responds to critical equipment alerts.	Aggregated cross-store data views, proactive anomaly detection such as freezer failures, and a clear summary of at-risk outlets.
Inventory & Supply Chain Lead	Reviews 7-day demand forecasts, approves or rejects restock recommendations, and monitors SKU stockout risks.	Accurate demand signals with confidence scores, clear explanations of what data drove each forecast, and integration with supply workflows.
Development Team	Builds and maintains the platform, AI prompting logic, Kafka pipelines, and REST API contracts.	A modular (Hexagonal) codebase with clear API contracts, stable AI structured outputs, and isolated domain logic for easy testing.
QA Team	Validates system behaviour, AI output accuracy, real-time data integrity, and edge case handling.	Defined request/response DTO formats, reproducible test cases, and coverage of edge cases such as Kafka lag and GLM hallucinations.

2. System Architecture & Design

2.1. High-Level Architecture

2.1.1. Overview

Type	Details
System	Web Dashboard (React 19 + Vite) and RESTful Backend (Spring Boot 3.4.1)
Architecture	Hexagonal Architecture (Ports & Adapters) with an Event-Driven Data Ingestion Layer (Apache Kafka)

FranchiseIQ is structured as a client-server system with a clear separation between the presentation layer, the application logic layer, and the infrastructure layer. The frontend is a single-page React 19 application served via Vite, which communicates with the Spring Boot backend through a versioned REST API. The backend itself follows the Hexagonal Architecture pattern, meaning that all business logic sits in the domain and application layers, while external dependencies such as the database, Kafka cluster, and the Z AI GLM API are treated as adapters that can be swapped independently.

Data enters the system through two pathways. The first is the standard REST API, which handles synchronous reads and writes from the React dashboard. The second is the Kafka pipeline, which continuously ingests POS transaction events from store terminals via the POST /events/pos/ingest endpoint. These two flows are intentionally decoupled so that a spike in transaction volume does not affect the responsiveness of the dashboard.

2.2. LLM as Service Layer

The Z AI GLM is not integrated as a generic "AI module" bolted onto the side of the system. Instead, it is embedded within the Application Use Case Layer as a dedicated reasoning service, accessed through LangChain4j, with clearly defined input contracts, output validation logic, and fallback behaviour. The key design principle guiding this integration is what the team calls the "Remove the AI" test: if the GLM is disconnected, all standard CRUD endpoints continue to function normally, and only the four AI Decision Node endpoints return an error state. This ensures the core platform is stable and testable independently of the AI provider.

2.2.1. Dependency Diagram (LLM Integration Flow)

The following steps describe how the GLM is integrated as a service layer within FranchiseIQ.

Step 1: Prompt Construction. When a manager triggers an AI-powered endpoint, such as POST /ai/pricing/optimize/{storeId}, the corresponding Use Case class in the application layer calls a LangChain4j prompt template. The template combines a system-level instruction (e.g., "You are a retail pricing analyst for MiXue Malaysia"), the manager's current trade-off weights (Cost Saving, Brand Presence, Revenue Maximisation), and the store's flattened sales DTO for the past 30 days.

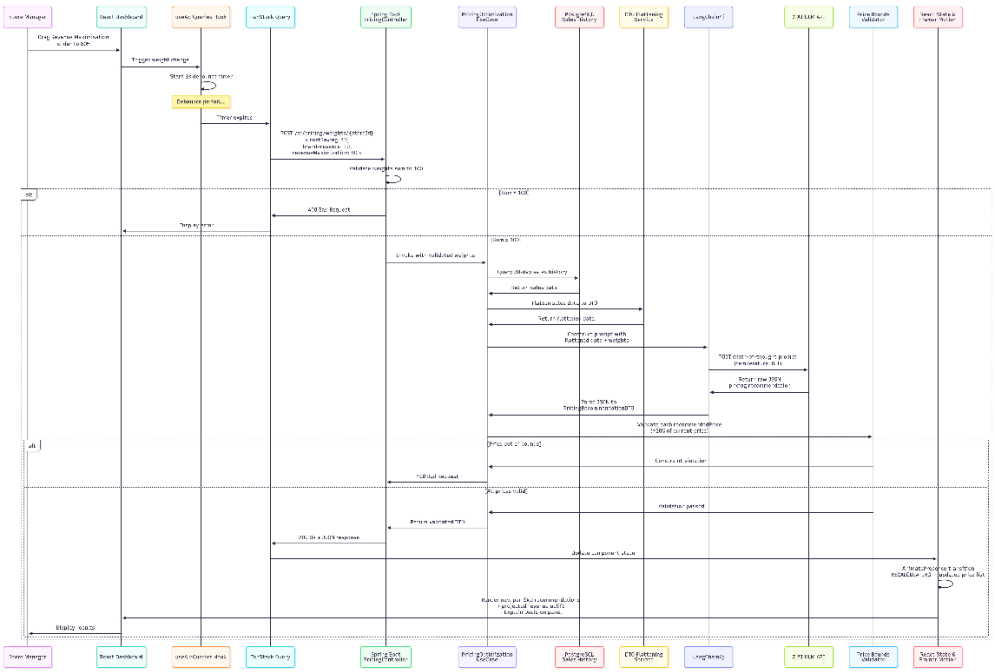
Step 2: Context Window Management. Before the prompt is sent, the system applies DTO Flattening, which strips all non-essential metadata and retains only high-density signals such as SKU identifiers, daily revenue figures, weather correlation scores, and sentiment cluster tags. The maximum context is capped at 128,000 tokens (Gemini 1.5 Pro backend support), with a rolling sales window logic that keeps only the most recent 100 transactions per SKU if the data volume exceeds the limit.

Step 3: Response Parsing. The GLM is instructed to return raw JSON only, with no markdown fencing, commentary, or preamble. LangChain4j's Structured Output Parser maps the JSON response directly to a Java DTO class (e.g., PricingRecommendationDTO). If the JSON is valid and passes schema validation, it is passed to the REST controller and serialised as the API response.

Step 4: Token Enforcement and Fallback. If the GLM returns malformed JSON or a response that fails schema validation, the system first retries with a "Strict Schema" correction prompt. If the retry also fails, the system falls back to the most recent stable decision cached in PostgreSQL and returns it with a "CACHED_RESULT" flag in the response body. On the frontend, this is rendered as an "AI Unavailable" badge rather than an error screen, ensuring the manager always sees actionable data.

Step 5: Major API Interactions. The React frontend communicates with the Spring Boot backend via the versioned base URL http://localhost:8080/api/v1. Standard CRUD endpoints such as GET /stores/{storeId} and GET /dashboard/kpis are served directly from the PostgreSQL database through JPA/Hibernate. AI Decision endpoints, prefixed with /ai/, pass through the Application Use Case Layer, which calls the Z AI GLM API via HTTPS using LangChain4j as the client wrapper. The Kafka Consumer Adapter runs as a background thread inside the Spring Boot process, continuously polling the POS transaction topic and writing events to PostgreSQL for persistence and dashboard display.

2.2.2. Sequence Diagram: Dynamic Pricing Recalculation Flow

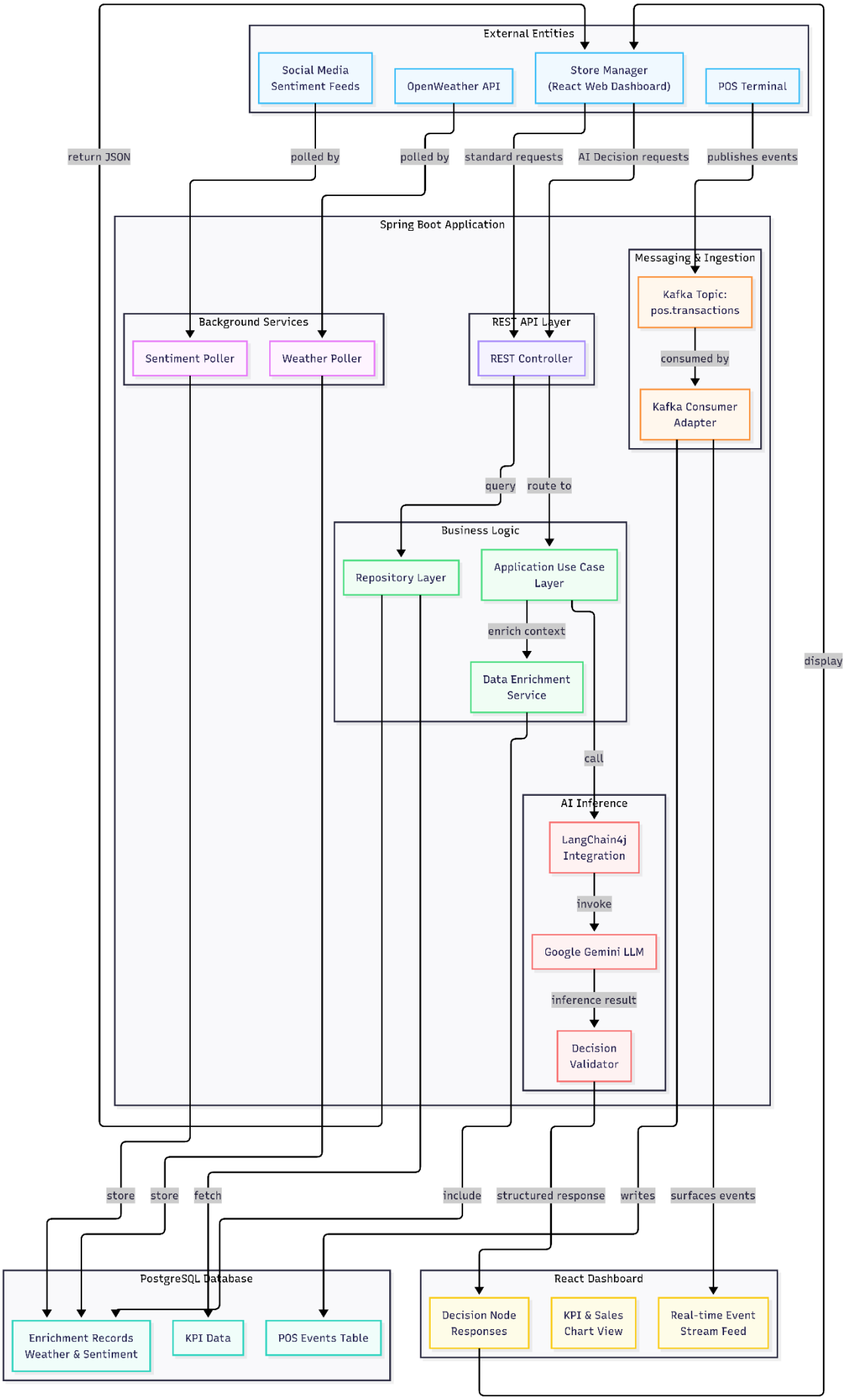


2.3. Technological Stack

Layer	Technology	Justification
Frontend	React 19 (Vite), Framer Motion, Recharts, TanStack Query v5	React 19 with Vite provides fast hot-module reload. TanStack Query manages server state, caching, and debounced mutations. Framer Motion handles AI loading state transitions.
Styling	TailwindCSS v4, Space Grotesk & Inter fonts	Utility-first CSS enables rapid iteration of the glassmorphism design system without custom stylesheet bloat.
Backend	Spring Boot 3.4.1 (Java 21/26), LangChain4j	Spring Boot provides a mature, production-grade REST framework. LangChain4j abstracts the GLM API, structured output parsing, and retry logic.
Database	PostgreSQL (JPA / Hibernate)	Relational database ensures transactional integrity for pricing decisions and cached AI outputs. JPA simplifies schema management.
Data Ingestion	Apache Kafka	Kafka decouples the high-throughput POS event stream from the AI inference layer, ensuring that transaction spikes do not degrade API response times.
AI Engine	Z AI GLM (FranchiseIQ-Sentiment-v2.3), Google Gemini (fallback)	Z AI GLM is selected for its high reasoning capability in financial trade-offs and ability to return strictly structured JSON without markdown. Gemini serves as the secondary fallback for sentiment analysis.
Testing	JUnit 5 (Backend), Vitest (Frontend), Mockito, Vi Mocks	Standard testing frameworks for each layer. Mockito and Vi Mocks isolate AI and database dependencies during unit and integration tests.

2.4. Key Data Flows

2.4.1. Data Flow Diagram (DFD)


External Entities
Social Media Sentiment Feeds
OpenWeather API
Store Manager (React Web Dashboard)
POS Terminal
return JSON
polled by
polled by
standard requests
AI Decision requests
publishes events
Spring Boot Application
Messaging & Ingestion
Kafka Topic: pos.transactions
consumed by
Kafka Consumer Adapter
Background Services
Sentiment Poller
Weather Poller
REST API Layer
REST Controller
query
route to
Business Logic
Repository Layer
Application Use Case Layer
enrich context
Data Enrichment Service
call
AI Inference
LangChain4j Integration
invoke
Google Gemini LLM
inference result
Decision Validator
display
store
store
fetch
include
structured response
writes
surfaces events
PostgreSQL Database
Enrichment Records Weather & Sentiment
KPI Data
POS Events Table
React Dashboard
Decision Node Responses
KPI & Sales Chart View
Real-time Event Stream Feed

2.4.2. Normalised Database Schema

The database is designed to Third Normal Form (3NF) to ensure data integrity and eliminate redundant data storage. The core entities are as follows.

The stores table is the central reference entity, identified by a store_id primary key, and holds location, tier, and operational status metadata. The skus table holds product master data, with each SKU referenced by a sku_id. The pos_events table records every transaction ingested from Kafka, with a foreign key to stores and a JSON payload column for raw item data. The pricing_recommendations table records every AI-generated pricing decision, with foreign keys to stores and skus, along with the weights used, the recommended price, and the timestamp. This table also serves as the cache for the fallback mechanism.

The sentiment_scores table stores the aggregated sentiment output per store per day, including the overall score, the breakdown by polarity, and the AI-generated summary text. The inventory_forecasts table stores 7-day demand predictions per SKU per store, with confidence scores and stockout risk classifications. The ai_decision_log table is a cross-cutting audit table that records every GLM inference call, including the model version used, the input token count, the response time, and whether the response passed schema validation or fell back to the cache. This table is critical for monitoring token costs and detecting hallucination patterns over time.

3. Functional Requirements & Scope

This section outlines the core boundaries of FranchiseIQ and establishes the features that will be built and demonstrated by the end of UMHackathon 2026. The scope is intentionally focused on a high-impact MVP that demonstrates the technical feasibility of the AI-native approach without overextending the team's capacity within the 10-day timeline.

3.1. Minimum Viable Product (MVP)

#	Feature	Description
1	**Dynamic Pricing (Human-in-the-Loop)**	Store managers adjust three trade-off sliders (Cost Saving, Brand Presence, Revenue Maximisation) that must sum to 100%. The frontend debounces the input by 1 second and sends the weights to the backend, which constructs a GLM prompt and returns a per-SKU price recommendation list with projected revenue uplift.
2	**Sentiment Scorecard**	The system aggregates customer reviews, Google Reviews, and equipment alerts (e.g., freezer temperature spikes) into a single 1-to-5 store health score. The GLM performs entity-level NLP analysis and generates a plain-language summary of the dominant sentiment themes, along with critical alerts such as food safety clusters.
3	**Inventory Forecasting**	The system produces a 7-day demand prediction for each active SKU in the store. The GLM cross-references historical POS data with OpenWeather weather forecasts, the school holiday calendar, and social media trends to generate a restocking recommendation with a confidence score and a stockout risk classification.
4	**Actionable Insights**	The GLM synthesises signals from all three other features and produces a priority-ranked list of plain-language recommendations for the store manager. Each insight includes an action, an estimated business impact in Ringgit, and a confidence score, allowing non-technical managers to act without needing to interpret the underlying data.
5	**Real-time Event Stream Feed**	A live ticker panel on the dashboard displays the most recent Kafka-ingested POS events across the store. Events are colour-coded by source (POS transaction, equipment alert, social mention) and update automatically, giving managers a real-time pulse of store activity.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Scalability	The system must handle high-traffic retail environments such as MiXue outlets with thousands of daily POS transactions without degrading API response times.	Kafka-based event ingestion is completely decoupled from the AI and REST layers. The Kafka consumer processes events asynchronously, ensuring that transaction bursts do not block synchronous API calls.
Reliability	AI pricing suggestions must remain within +/- 20% of the current price for any given SKU, ensuring	Backend PriceValidator service applies hard-coded pricing bounds to every GLM output before it is returned. If the recommended price falls outside the

Quality	Requirement	Implementation
	managers are never presented with nonsensical recommendations.	bounds, it is clamped and flagged in the response.
Token Latency	AI re-analysis must complete within 3 to 5 seconds at the 95th percentile (p95) under normal load conditions.	Debounced inputs (1-second delay) prevent redundant GLM calls. The HTTP client is configured with a 180-second timeout. The React frontend displays an AnimatePresence-driven RECALCULATING badge during the wait, so the manager is never left with a blank screen.
Cost Efficiency	LLM API costs per manager session must remain at or below an estimated USD 0.05, making the system viable at scale across hundreds of franchise outlets.	React Query caches AI responses for 24 hours on static data such as sentiment baselines. Debounced recalculations prevent redundant API calls. DTO Flattening minimises token consumption per request by stripping non-essential data before injection.
Maintainability	The codebase must be structured such that a new AI feature or a new data source can be added without modifying existing domain logic.	Hexagonal Architecture ensures that all new AI use cases are added as independent classes in the application layer, with no changes required to domain entities or existing adapters. LangChain4j prompt templates are stored as versioned configuration files.

3.3. Out of Scope / Future Enhancements

The following features were identified during the planning phase as part of the team's longer-term vision for FranchiseIQ, but are deliberately excluded from the UMHackathon 2026 scope to maintain focus on the AI-native MVP.

- Real-time chat or messaging between store managers, regional supervisors, or between users and the AI system. This is a significant scope item that would require a separate WebSocket or Supabase Realtime integration layer.
- A production-ready payment gateway integration for processing financial transactions or subscription billing within the platform.
- A native mobile application for iOS or Android. The initial version of FranchiseIQ is web-only, and the responsive design is optimised for desktop dashboards used by store managers on-site.
- Multi-store comparison dashboards and aggregate analytics across the full franchise network, which would require additional data modelling and cross-tenant access controls.
- Automated supplier order placement triggered by inventory forecast recommendations, which would require integration with external supply chain APIs.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Grayscale Rollout and A/B Testing

New AI Strategy Modules, such as the planned "Clearance Sale" pricing mode, are released to 5% of stores initially through a feature flag mechanism in the Spring Boot configuration layer. During this observation window, the team monitors the rate of invalid JSON responses from the GLM, the p95 API latency, and the manager interaction rate with the new module's Decision Nodes. If no critical issues are detected after 48 hours, the rollout percentage is increased to 25%, then 50%, and finally 100%. A/B testing is then conducted to compare the revenue impact on stores using the new module against a control group still using the previous strategy, with the winning variant becoming the new default.

4.1.2. Emergency Rollback and Golden Release

The emergency rollback mechanism is triggered automatically when the rate of invalid JSON responses from the GLM exceeds 1% of total inference calls within a 15-minute rolling window. In this event, the system reverts all AI Decision endpoints to a "Heuristic-based" pricing fallback, which uses the most recent validated recommendation stored in the PostgreSQL pricing_recommendations table. The previous stable deployment, referred to as the Golden Release, is kept in a tagged Docker image repository and can be re-deployed via the CI/CD pipeline within approximately 5 minutes. The React frontend renders an "AI Unavailable" notice rather than an error screen, ensuring managers retain access to historical data while the GLM issue is being resolved.

4.1.3. Priority Matrix

Priority	Trigger Condition	Threshold	Action
P1 Critical	GLM returns invalid JSON schema	> 1% of calls in 15 min	Emergency rollback to Golden Release; activate heuristic-based pricing fallback.
P1 Critical	AI pricing output exceeds sanity bounds	Any single SKU	PriceValidator clamps the price; logs incident to ai_decision_log; notify Dev Team.
P2 High	Kafka consumer lag exceeds threshold	> 10,000 messages	Alert Dev Team; scale Kafka consumer instances; investigate producer throughput.
P2 High	AI response latency spike	p95 > 8 seconds	Activate cached response mode; reduce context window size by 30%; notify users.
P3 Medium	PostgreSQL connection pool exhaustion	> 90% pool usage	Alert Dev Team; scale database connection pool; review slow query log.

4.2. Monitoring

The monitoring plan for FranchiseIQ covers three layers. At the infrastructure layer, the team uses standard Spring Boot Actuator endpoints (/actuator/health and /actuator/metrics) to expose JVM memory usage, active thread counts, and database connection pool statistics. These are polled every 30 seconds by the CI/CD monitoring dashboard.

At the application layer, every AI inference call is logged to the ai_decision_log table with the model version, input token count, response latency, and a pass/fail flag for schema validation. A scheduled Spring Boot task aggregates this log hourly and calculates the rolling invalid JSON rate, the average token cost per session, and the p95 latency. If any metric crosses its P1 threshold, a notification is sent to the on-call developer.

At the Kafka layer, consumer group lag is monitored via the Kafka AdminClient API. If the lag for the pos.transactions consumer group exceeds 10,000 messages, an automated alert is raised. During the hackathon, this monitoring is simulated using a mock producer that generates 100 POS events per second to validate that the consumer keeps pace without blocking the REST API.

4.3. Assumptions

The following key assumptions were made during the design and development of FranchiseIQ.

- Store managers and supervisors have access to a stable internet connection while using the web dashboard, as the system does not support an offline or progressive web app mode in this version.
- POS systems at MiXue outlets are capable of pushing transaction data to the /events/pos/ingest endpoint via HTTP webhooks or are connected to the Kafka cluster through a compatible producer client.
- The Z AI GLM API maintains the structured JSON output format as documented in this SAD. Any breaking changes to the GLM's output schema would require updates to the LangChain4j structured output parser.
- Managers have a basic familiarity with the concept of trade-off sliders and are comfortable making pricing decisions based on AI recommendations, provided those recommendations are accompanied by plain-language explanations.
- The OpenWeather data sources remain accessible under their current free or trial tier quotas for the duration of the hackathon.

4.4. External Dependencies

Tool / Service	Purpose	Risk & Mitigation
Z AI GLM API	Core reasoning engine for all four AI Decision Nodes. Handles multi-step chain-of-thought inference and generates structured JSON output for pricing, sentiment, inventory, and insights.	High. Rate limits or API downtime would disable all four AI features. Mitigation: Graceful degradation to cached PostgreSQL results and a visible "AI Unavailable" state in the UI.

Tool / Service	Purpose	Risk & Mitigation
Google Gemini API	Secondary fallback engine for sentiment analysis when the Z AI GLM is unavailable or exceeds its rate limit.	Medium. Used only as a fallback, so its unavailability does not trigger an emergency rollback. Mitigation: Sentiment falls back to the last cached score.
Apache Kafka Cluster	High-throughput POS event streaming from store terminals into the FranchiseIQ backend. Decouples transaction ingestion from the AI inference layer.	Medium. Consumer lag could delay event stream display. Mitigation: Asynchronous processing, consumer group monitoring, and auto-scaling of consumer instances.
OpenWeather API	Provides weather forecast data used as a contextual signal in inventory demand prediction and pricing strategy during extreme weather events.	Low. Weather data is enrichment, not a core dependency. Mitigation: If the API is unavailable, the inventory forecast uses only historical POS data with a reduced confidence score.
Supabase (PostgreSQL)	Managed PostgreSQL database for persisting all store data, AI decision logs, Kafka event records, and cached AI recommendations.	Low. Supabase provides a managed, high-availability PostgreSQL instance. Mitigation: Connection pooling via JPA/Hibernate; Spring Boot retries on connection failure.

5. Project Management & Team Contributions

The UMHackathon 2026 preliminary round spans approximately 10 days, which the team structured as a focused two-week sprint. The timeline below reflects the team's actual development sequence, with each phase building directly on the previous one to ensure that the AI integration had a stable backend and schema to work against before the frontend was layered on top.

5.1. Project Timeline

Phase	Days	Deliverables
Phase 1: Foundation	Day 1 to 3	Core Hexagonal Architecture setup, PostgreSQL schema design (3NF), Kafka topic configuration, and initial Spring Boot project scaffolding. REST API contracts defined and shared with the frontend team.
Phase 2: AI Integration	Day 4 to 6	LangChain4j integration with Z AI GLM, prompt template design for all four use cases, DTO flattening service, schema validation and retry logic, and PostgreSQL caching for AI outputs.
Phase 3: Frontend & Sync	Day 7 to 9	React 19 dashboard development including all four feature panels, TanStack Query hooks, Framer Motion loading states, trade-off slider with auto-balancing logic, and Kafka event stream feed integration.
Phase 4: Testing & Deployment & Docs	Day 10	JUnit 5 and Vitest test suite execution, QA validation of all four test cases (TC-01 to TC-04), final documentation of SAD and PRD, and deployment preparation.

5.2. Team Members and Roles

Member	Role	Key Responsibilities
Backend Lead – Too Tze Jiat	Backend Architect & AI Engineer	Hexagonal Architecture design, Spring Boot project setup, LangChain4j integration, Z AI GLM prompt engineering, DTO flattening service, schema validation, and JUnit 5 test suite.
Frontend Lead – Cindy Pua Kah Qi	Frontend Engineer & UX Designer	React 19 dashboard architecture, component design for all four feature panels, TanStack Query hooks and caching strategy, Framer Motion animation system, trade-off slider implementation, Vitest test suite
DevOps Lead – Shawn Ryan Peterson	DevOps Engineer & CI/CD Engineer & System Architect	Design system architect, Setup HTTP proxy, Setup domain names, Dockerfiles, Service discovery, health checks, Resource Limits, Deployment, Multi-stage Builds, Kafka consumer configuration, and Kafka event stream feed.

UM Hack Tasks

Table

Task	Owner	Priority	Label	Notes
Project Skeleton Setup	Too Tze Jiat	Done	Architecture	Initialize Spring Boot 3.4.1, Java 21, and project modules.
Hexagonal Ports & Adapters	Too Tze Jiat	Done	Architecture	Define application.port.in and application.port.out packages.
Domain Model Definition	Too Tze Jiat	Done	Domain	Create Records/Entities for StockLevel, PricingStrategy, and SentimentScore.
LangChain4j Service C OPEN	Too Tze Jiat	Done	AI	Setup AIConfig.java with Z AI GLM (llm.x-ai/glm-4.6v-flash)
Prompt Engineering (GLM)	Too Tze Jiat	Done	AI	Design system prompts for Multi-step Agentic Prompting.
DTO Flattening Service	Too Tze Jiat	Done	Utility	Logic to strip high-density data for LLM context window (128k).
PostgreSQL Schema Design	Too Tze Jiat	Done	DB	Define tables for Sales history, SKU pricing, and cached AI decisions.
Price Sanity Validator	Too Tze Jiat	Done	Logic	Implement +/- 20% hard-coded bounds check for AI pricing outputs.
JUnit 5 Test Suite	Too Tze Jiat	Done	QA	Mock GLM API to test parsing logic and DTO flattening.
React 19 Shell Setup	Cindy Pua	Done	Frontend	Setup Vite + Tailwind CSS v4 + Space Grotesk/Inter fonts.
TanStack Query Provider	Cindy Pua	Done	Logic	Configure global caching and staleTime strategy for AI endpoints.
Dynamic Pricing Slider	Cindy Pua	Done	Component	Implement trade-off slider with debounced (1s) auto-balancing logic.
Inventory Forecast Panel	Cindy Pua	Done	Component	Build predictive stock view with 'Explain This Decision' modal.
Sentiment Re-analysis Card	Cindy Pua	Done	Component	Create health scorecards (1-5) with Framer Motion animations.
Kafka Live Ticker	Cindy Pua	Done	Component	Build the 'heartbeat' feed for real-time POS transaction events.
Vitest Suite Setup	Cindy Pua	Done	QA	Write unit tests for slider logic and optimistic UI updates.
Multi-stage Dockerfiles	Shawn Ryan Peterson	Done	Docker	Optimize Spring Boot (JRE 21 Alpine) and Nginx images.
Kafka Broker Setup	Shawn Ryan Peterson	Done	Ingestion	Setup local/cloud Kafka clusters and define pos-transactions topic.
Kafka Consumer Config	Shawn Ryan Peterson	Done	Ingestion	Implement consumer logic in Spring Boot to ingest POS data.
CI/CD Pipeline (GH Actions)	Shawn Ryan Peterson	Done	DevOps	Automate Lint -> Test -> Build -> Deploy flow.
Reverse Proxy & Domain	Shawn Ryan Peterson	Done	Network	Configure Nginx for retailers.webhop.me and handle CORS/SSL.
Resource Limit Policy OPEN	Shawn Ryan Peterson	Done	Cloud	Set CPU/RAM limits for AI containers to prevent Kafka lag.
Service Discovery & Health	Shawn Ryan Peterson	Done	DevOps	Configure Spring Boot Actuator for container health checks.
Mock POS Stream Producer	Shawn Ryan Peterson	Done	Testing	Script to push 10k+ mock events to test Kafka throughput.

5.3. Recommendations for Future Development

Based on the architecture decisions made during this hackathon, the team offers the following recommendations for scaling FranchiseIQ beyond the current MVP.

The first recommendation is to introduce a Redis cache layer in front of the Z AI GLM API. Currently, React Query handles client-side caching for 24 hours, but a server-side Redis cache with a 6-hour TTL for pricing recommendations would reduce redundant GLM calls across multiple concurrent user sessions accessing the same store's data, significantly lowering token costs at scale.

The second recommendation is to migrate the Kafka deployment from the current single-node mock setup to a managed Kafka cluster (such as Confluent Cloud or AWS MSK) once the system is deployed across more than 10 stores. This would provide replication, fault tolerance, and consumer group rebalancing out of the box, without requiring the team to manage Kafka infrastructure manually.

The third recommendation is to implement a dedicated AI Observability layer, potentially using LangSmith or a similar LLM tracing tool, that captures the full prompt, context window content, and response for every inference call. This would make it significantly easier to debug pricing anomalies, detect prompt regression when the GLM model is updated, and audit AI-driven decisions for compliance purposes.

The fourth recommendation is to extend the Hexagonal Architecture by introducing a dedicated Port interface for the AI reasoning engine, which would allow the team to swap the Z AI GLM for any other LLM provider (such as OpenAI GPT or Anthropic Claude) by implementing a new Adapter class, without modifying any use case or domain logic.

6. Quality Assurance Testing Documentation (QATD)

6.1. Objective

The primary objective of the FranchiseIQ QA process is to ensure that the system reliably handles real-time POS data ingestion, AI-driven pricing recalculations, and sentiment scoring under concurrent load, with automated CI/CD checkpoints that prevent regression. Every test is designed to validate not just that the system produces a response, but that the response is structurally valid, within acceptable business bounds, and delivered within the performance targets defined in the NFRs.

6.2. Scope and Requirements Traceability

The in-scope features for testing are the AI Pricing Use Case (weight-based optimisation and real-time recalculation), the Sentiment Tracking Use Case (review and equipment alert analysis), and the Kafka Ingestion pipeline (high-throughput transaction streaming from /pos/ingest). Out of scope for this test cycle are multi-user real-time collaboration and production-grade payment gateway processing, as these are explicitly excluded from the MVP scope.

6.3. Risk Assessment and Mitigation Strategy

Technical Risk	Likelihood (1-5)	Severity (1-5)	Risk Score	Mitigation Strategy	Testing Approach
AI Hallucination in Pricing	**2**	**5**	**10 (Med)**	Implement backend sanity bounds (+/- 20% limit) validated by PriceValidator service.	Unit tests for PriceValidator service with boundary-value inputs.
Kafka Topic Backlog	**3**	**4**	**12 (High)**	Asynchronous processing and consumer group lag monitoring with auto-scaling.	Load testing with 10,000+ mock POS events ingested within 1 second.
Invalid JSON from GLM	**3**	**5**	**15 (High)**	LangChain4j Structured Output Parsers with retry logic and a schema correction prompt.	Integration tests using malformed JSON mocks to validate parser resilience.

6.4. Test Environment and Execution Strategy

Unit tests cover Java domain logic and React utility functions. They are executed on every commit via JUnit 5 on the backend and Vitest on the frontend, using Mockito and Vi Mocks to isolate AI and database dependencies from the logic being tested.

Integration tests cover controller-to-agent communication. They are executed using @WebMvcTest with MockMvc on the backend, with the physical GLM API mocked to test the parsing logic and DTO flattening service against both valid and malformed response payloads.

The passing rate thresholds are set at 100% for the critical path (Pricing Recalculation and POS Ingestion) and 90% overall coverage for the core application layer. Any failure in the critical path will block the CI/CD pipeline from proceeding to deployment.

6.5. Test Case Specifications

ID	Type	Description	Expected Result
TC-01	Happy Case: Dynamic Recalculation	User drags the Revenue Maximisation slider to 80% and pauses for 1 second. The debounce timer expires and the mutation is triggered via TanStack Query.	The AI returns a valid pricing recommendation JSON. The UI updates with the new per-SKU prices and the projected revenue uplift amount. The RECALCULATING badge disappears within 5 seconds.
TC-02	Negative: Invalid Weight Sum	User attempts to manually bypass the slider auto-balance logic and sends a request with weights totalling 150%.	The UI auto-balances to 100% before the API call is made. The backend returns 400 Bad Request for any payload where the weights do not sum to exactly 100.
TC-03	NFR: AI Response Latency	Trigger an AI pricing recalculation 20 times consecutively under simulated concurrent load to measure p95 response time.	p95 response time is under 5 seconds. The RECALCULATING badge is visible during the entire wait period. No response returns null or an empty recommendations array.
TC-04	NFR: Kafka Throughput	Use the mock Kafka producer to ingest 100 POS events within 1 second via the /events/pos/ingest endpoint.	The backend returns 202 Accepted immediately for all 100 events. All events appear in the Event Stream Feed on the dashboard within 3 seconds. No events are dropped or duplicated.

6.6. AI Output and Boundary Testing

6.6.1. Prompt and Response Test Pair

Input: "Optimise pricing for store-kl-001 with 80% Cost Saving priority, 10% Brand Presence, and 10% Revenue Maximisation."

Acceptance Criteria: The returned JSON must contain a recommendations list where every SKU's recommendedPrice is less than or equal to its currentPrice, correctly reflecting the cost-saving priority. The projectedRevenue object must show a lower uplift than would be expected from a Revenue Maximisation strategy.

6.6.2. Oversized Input Handling

The system is tested with store data sets that exceed 30 days of sales history. The expected behaviour is that the rolling window logic automatically truncates the input to the most recent 100 transactions per SKU, ensuring the context window stays within the 128,000-token limit. The test validates that the truncation does not affect the structural validity of the prompt and that the GLM still returns a valid response.

6.6.3. Hallucination Handling

The Z AI GLM is configured with a temperature of 0.1 to maximise output determinism and reduce the likelihood of creative or unpredictable responses. Post-inference, the backend performs a schema validation check on the raw JSON response. If the schema is invalid, the system retries once with a strict schema correction prompt. If the retry also fails, the system falls back to the most recent stable pricing decision stored in PostgreSQL, which is returned with a CACHED_RESULT flag in the API response. This entire flow is tested by injecting a mocked GLM response that deliberately violates the DTO schema and verifying that the fallback mechanism activates correctly.